Java加强

zcx

一、多线程

- 线程(Thread)是一个程序内部的一条流程
- 如果程序中只有一条执行流程，那这个程序就是单线程程序
- 多线程是指从软硬件上实现的多条执行流程的技术(多条线程由CPU负责调度执行)
- 用途：如多人同时购票，上传的同时也在下载，消息通信

1. **创建线程**
   - ***方法一：继承Thread类***，则为子类，该子类需重写Thread类的run方法，然后分配和启动子类的实例
   -
     ```
     public class demo1 {
     public static void main(String[] args) {
     myThread t = new myThread(); 声明子类对象
     t.start(); 启动线程
     System.out.println("main主线程");
     } 其实main方法就是一条线程，然后对象t又启动了一个线程，现在是多线程，会有异步性，即主函数和子类对象中的输出会异步，即独立并发的执行
     }
     class myThread extends Thread{
     @Override
     public void run() { 重写run方法
     System.out.println("线程启动了");
     }
     }这种方法，子类无法继承其他类，不利于功能的扩展
     ```
   - 注意事项：**启动线程必须调用start方法，调用run方法会当做普通方法执行，不会异步了，此时相当于单线程。不要把主线任务放在启动子线程之前，这样会导致主线程先跑完，然后跑子线程，没有异步了**
   - ***方法二：使用实现Runnable接口，创建线程***。即创建一个实现类实现Runnable接口，并重写run方法，接着创建线程任务类的对象，因为start是线程的方法，不是线程任务的方法，所以无法直接用线程任务类对象调用start方法，所以只能把任务对象包装成线程对象，然后调用start方法启动线程
   -
     ```
     public class demo2 {
     public static void main(String[] args) {
     ```

```
myrunnable r = new myrunnable(); 声明实现类对象
Thread t = new Thread(r); 把任务对象包装成线程对象
t.start(); 启动线程
}
}
class myrunnable implements Runnable{ 实现Runnable接口
@Override
public void run() { 重写run方法
System.out.println("多线程创建");
}
}
```

- 优缺点：优点：任务类是实现接口，可以继承其他类，实现其他接口，扩展性强。缺点是需要多一个Runnable对象
- 方法二还有匿名内部类写法，只是简化写法
- ```
  //匿名内部类写法
  new Thread(new Runnable() {
  @Override
  public void run() {
  System.out.println("多线程创建");
  }
  }).start();
  ```
- 同样可以有lambda表达式写法
- ```
  new Thread(() -> System.out.println("多线程创建")).start();
  ```
- 前两种方法都有一个问题，即重写run方法后，因为是void类型，所以无法返回结果
- ***方法三：实现Callable接口，创建线程***：通过Callable接口和FutureTask类解决返回结果的问题，即可以返回线程执行完毕后的结果
- 步骤：
  - 创建任务对象：定义一个类实现Callable接口，重写call方法，封装要做的事情和要返回的数据。然后要把Callable类型的对象包装成FutureTask对象(线程任务对象)，
  - 把线程任务对象交给Thread对象
  - 调用Thread对象的start方法启动线程
  - 线程执行完毕后，通过FutureTask对象调用get方法获取线程执行结果
- ```
  import java.util.concurrent.Callable;
  import java.util.concurrent.ExecutionException;
  import java.util.concurrent.Future;
  import java.util.concurrent.FutureTask;
  public class demo3 {
  public static void main(String[] args) throws ExecutionException, InterruptedException {
  mycallable c = new mycallable(); 声明实现类对象c
  ```
  ```

```
FutureTask< String> ft = new FutureTask< String>(c); 把实现类对象包装成线程任务对象ft
Thread t = new Thread(ft); 把线程任务对象ft交给Thread对象t
t.start(); 启动线程
System.out.println(ft.get()); 用ft.get()方法获取线程执行结果,因为是线程任务
}
}
定义一个实现类实现Callable接口
class mycallable implements Callable< String> {
@Override
public String call() throws Exception { 重写call方法
int sum = 0;
for(int i = 1; i <= 100; i++){
sum += i;
}
return "子线程计算后的结果为："+ sum; 返回结果
}
}
```

- 如果第一个线程发现该线程对象未结束，则会让权等待，主线程暂停，直到第一个线程执行完毕才会接着往下走
- 第三个方法的优缺点：优点：线程任务类只是实现接口，可以继续继承类和实现接口，扩展性强，可以在线程执行完毕后去获取线程执行的结果。缺点：编码相对复杂

2. **线程的常用方法**

-

Thread提供的方法	作用
start()	启动线程
run()	线程任务方法
sleep(long millis)	线程休眠
getName()	获取线程名称
setName(String name)	设置线程名称
currentThread()	获取当前线程对象
join()	调用这个方法的线程先执行，即线程插队

-

Thread类提供的构造器	作用
Thread()	创建线程对象
Thread(Runnable target)	封装Runnable对象为线程对象

Thread类提供的构造器	作用
Thread(Runnable target, String name)	封装Runnable对象为线程对象，并设置线程名称
Thread(String name)	创建线程对象，设置线程名称

-
```
import static java.lang.Thread.currentThread;
public class demo1 {
public static void main(String[] args) {
//学习线程提供的方法
mythread t1 = new mythread(); 创建线程对象t1
t1.start();
mythread t2 = new mythread(); 创建线程对象t2
t2.start();
System.out.println(t1.getName()); 打印结果为Thread-0
System.out.println(t2.getName()); 打印结果为Thread-1
Thread T = currentThread(); 哪个线程调用这行代码，就返回哪个线程
//这是主函数调用，所以下面一行打印结果为main
System.out.println(T.getName());
mythread t3 = new mythread(); 创建线程对象t3
t3.setName("线程3"); 在线程启动前设置名字
t3.start();
System.out.println(t3.getName()); 打印结果为线程3
mythread t4 = new mythread("4号线程"); 创建线程对象t4，并设置线程名称
t4.start();
System.out.println(t4.getName()); 打印结果为4号线程
Runnable r = new myrunnable(); 创建任务对象r
Thread t5 = new Thread(r,"5号线程"); 创建线程对象t5，并设置线程名称
Thread t6 = new Thread(r); 创建线程对象t6
}
}
class mythread extends Thread{
public mythread(String name) { 有参构造器
super(name); 调用父类的有参构造器，设置线程名称
}
public mythread() { 无参构造器
}
@Override
public void run() { 重写run方法，线程任务方法
System.out.println(Thread.currentThread().getName()+"多线程创建");
哪个线程调用，就返回哪个线程的名称+多线程创建这行字符串
```

```
}
}
class myrunnable implements Runnable{ 创建任务类
@Override
public void run() { 重写run方法，线程任务方法
System.out.println("多线程创建");
}
}
public static void main(String[] args) {
搞清楚Thread的sleep方法
for(int i = 0; i < 100; i++){
System.out.println("多线程创建");
try {
Thread.sleep(1000); 线程睡眠1秒
System.out.println("线程睡眠1秒");
}catch (InterruptedException e) {
e.printStackTrace();
}
}
} 每次循环都会睡眠1秒，再继续
public static void main(String[] args) {
搞懂join()方法
for(int i = 0; i < 12; i++){ 循环12次
System.out.println("多线程创建");
if(i == 9){ 当i等于9时，创建线程t，并启动线程t
Thread t = new Thread(() -> {
for(int j = 0; j < 10; j++){
System.out.println("多线程创建iiii");
} t线程执行10次
});
t.start(); 启动线程t
try {
t.join(); 调用t线程的join()方法，当前线程会等待t线程执行完毕
System.out.println("t线程执行完毕");
}catch (InterruptedException e) {
e.printStackTrace();
}
} 等待t线程执行完毕后继续往下外层循环
}
}
```

3. **线程安全**
   - 多个线程，同时操作同一个共享资源的时候，可能会出现业务安全问题
4. **线程同步**
   - 解决线程安全问题的解决方案
   - 核心思想：让多个线程先后依次访问共享资源
   - 常见方案：**加锁**：每次只允许一个线程加锁，加锁后才能进行访问，访问完成后释放锁，其他线程才能再加锁继续访问
   - **方式一：同步代码块**，
     - 作用：把访问共享资源的核心代码给上锁(**sychronized**)，保证线程安全。
     - 原理：每次只允许一个线程加锁后进入，执行完毕后自动解锁，其他线程才能进入。
     - 同步锁注意事项：对于当前同时执行的线程来说，同步锁必须是同一把(同一个对象，唯一一把)
     -
       ```
       public class account {
       private String accountNo;
       private double balance;
       public void drawoney(double drawAmount){
       String name = Thread.currentThread().getName();
       synchronized ("dlei") { 把访问共享资源的核心代码给上锁,设置了唯一的锁"dlei"
       if (balance >= drawAmount) {
       balance -= drawAmount;
       System.out.println(name + "取钱成功，余额为：" + balance);
       } else {
       System.out.println(name + "取钱失败，余额不足");
       }
       } 同步代码块结束
       }
       }
       "dlei"这种锁范围太大了，因为如果有多个账户对象，都要取钱，所有的账户对象都会被锁住，但是其实时要求每个账户对象的多个线程要同步锁
       所以可以修改这个锁，为this，这样多个账户对象之间互不影响
       建议使用共享资源作为锁对象，对于实例方法建议使用this作为锁对象,对于静态方法建议使用字节码(类名.class)对象作为锁对象
       ```
   - **方式二：同步方法**
     - 作用：把访问共享资源的核心方法给上锁
     - 底层原理：同步方法底层有隐式锁对象，锁的范围为整个方法代码，如果方法是实例方法，默认为this作为锁的对象，如果是静态方法，默认为字节码对象(类名.class)
     -
       ```
       public class demno2 {
       private String accountNo;
       private double balance;
       public synchronized void drawoney(double drawAmount){ 把访问共享资源的核心方
       ```

```
法给上锁,在方法上加synchronized
String name = Thread.currentThread().getName();
if (balance >= drawAmount) {
balance -= drawAmount;
System.out.println(name + "取钱成功，余额为：" + balance);
} else {
System.out.println(name + "取钱失败，余额不足");
}
} 同步方法结束
}
```

- **方法三：Lock锁**
  - 通过Lock锁可以创建出锁对象进行加锁和解锁，更灵活，方便，强大
  - Lock是接口，不能直接实例化，可以用他的实现类ReentrantLock来构建Lock锁对象
  -

方法	描述
ReentrantLock()	创建一个Lock锁的实现类对象
lock()	加锁
unlock()	解锁
tryLock()	尝试加锁

  -

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
public class demo3 {
private String accountNo;
private double balance;
private final Lock lk = new ReentrantLock(); 创建一个Lock锁的实现类对象lk，使用final修饰防止被被人修改
public void drawoney(double drawAmount){
String name = Thread.currentThread().getName();
lk.lock(); 上锁
try{ try-catch-finally
if (balance >= drawAmount) {
balance -= drawAmount;
System.out.println(name + "取钱成功，余额为：" + balance);
} else {
System.out.println(name + "取钱失败，余额不足");
}
}finally { 保证释放锁，从而避免发生死锁
lk.unlock(); 释放锁
```

```
}
}
}
```

5.**线程池**

- 线程池是一个可以复用线程的技术
- 不使用线程池的问题：用户每发起一个请求，后台就需要创建一个新线程来处理，下次新任务来了又要创建，新线程的开销很大，而且请求过多时，会产生大量的线程，会严重影响系统的性能
- 工作原理：少量的工作线程处理一堆任务队列。(相当于一些服务员服务成百的顾客一样)‘
- **线程池的创建**：代表线程池的接口：ExecutorService
  - 如何创建线程池对象：
  - 方式一：使用ExecutorService接口的实现类ThreadPoolExecutor自创建一个线程池对象
  -

参数	描述
corePoolSize	线程池中的核心线程数(正式工)
maximumPoolSize	线程池中的最大线程数(正式工加临时工)
keepAliveTime	指定临时线程存活时间(临时工空闲多久被开除)
unit	指定临时线程存活的时间单位
workQueue	存放任务的队列(客人排队的地方，即等待执行的地方)
threadFactory	创建线程的工厂(招聘员工的HR)
handler	任务拒绝策略(线程都在忙，任务队列也满了，新任务来了怎么办)

  -
```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.Executors;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;
public class executorservice {
public static void main(String[] args) {
//使用线程池的实现类ThreadPoolExecutor声明7个参数来创建
ThreadPoolExecutor executor = new ThreadPoolExecutor( 创建线程池
corePoolSize: 3 ,
maximumPoolSize: 5,
keepAliveTime: 1000,
unit: TimeUnit.MILLISECONDS,
workQueue: new ArrayBlockingQueue< Runnable >(3),
threadFactory: Executors.defaultThreadFactory(),
handler: new ThreadPoolExecutor.AbortPolicy());
```

```
}
}
```

- 方式二：使用Executors(线程池的工具类)调用方法返回不同特点的线程池对象
-

方法名	描述
newFixedThreadPool(int nThreads)	创建一个固定大小的线程池， 如果某个线程因为执行异常结束， 则线程池会补充一个新线程替代它
newCachedThreadPool()	创建一个可缓存的线程池， 线程数量随着任务的增加而自动增加线程数， 若线程任务执行完毕空闲了60s，则回收
newSingleThreadExecutor()	创建只有一个线程的线程池， 如果这个线程因为执行异常结束， 则线程池会补充一个新线程替代它
newScheduledThreadPool(int corePoolSize)	创建一个定长线程池，支持定时及周期性任务执行

-

```
public static void main(String[] args) {
//通过创建Executors类中的方法获取线程池对象
ExecutorService pool1 = Executors.newFixedThreadPool(5);
ExecutorService pool2 = Executors.newSingleThreadExecutor();
ExecutorService pool3 = Executors.newCachedThreadPool();
ExecutorService pool4 = Executors.newScheduledThreadPool(5);
}
```

- **executors使用可能存在的缺陷**：大型并发系统环境中使用Executors如果不注意可能会出现系统风险，如FixedThreadPool和SingleThreadExecutor允许请求队列长度为Integer.MAX_VALUE，可能会导致OOM，CachedThreadPool和ScheduledThreadPool允许创建线程数为Integer.MAX_VALUE，可能会导致OOM

- **利用线程池处理Runnable任务**：用ExecutorService的常用方法
-

方法名	描述
void execute(Runnable command)	提交一个Runnable任务
Future<?> submit(Callable< T > task)	提交一个Callable任务，返回一个Future对象， 用于获取任务执行结果
void shutdown()	等全部任务执行完毕后，关闭线程池

方法名	描述
List< Runnable > shutdownNow()	中断所有正在执行的任务，并关闭线程池，并返回尚未执行的任务列表

-
```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.Executors;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;
public class executorservice {
public static void main(String[] args) {
//使用线程池的实现类ThreadPoolExecutor声明7个参数来创建
ThreadPoolExecutor TE = new ThreadPoolExecutor( 声明线程池对象TE
3,5,
1000, TimeUnit.MILLISECONDS,
new ArrayBlockingQueue(3),
Executors.defaultThreadFactory(),
new ThreadPoolExecutor.AbortPolicy());
myrunnable r = new myrunnable(); 创建任务对象r
TE.execute(r); //提交第一个任务 线程池会创建一个线程执行
TE.execute(r); //提交第二个任务 线程池会创建一个线程执行
TE.execute(r); //提交第三个任务 线程池会创建一个线程执行
TE.execute(r); //复用，因为corePoolSize为3
TE.execute(r); //复用，因为corePoolSize为3
//一般不关闭线程池，如下只是演示
TE.shutdown(); //等待所有任务执行完毕再关闭
TE.shutdownNow(); //立即关闭，并打断正在执行的任务
}
}
public class myrunnable implements Runnable{ 线程任务类
@Override
public void run() {
for(int i = 0; i < 5; i++){
System.out.println(Thread.currentThread().getName()+"输出"+ i);
}
}
}
```

- **线程池的注意事项**
  - 什么时候开始创建临时线程：新任务提交时，发现核心线程都在忙，任务队列也满了(等待执行的任务数等于workQueue容量)，并且还可以创建临时线程(maximumPoolSize有容量)，此时才会创建临时线程’

- 什么时候会拒绝任务：核心线程和临时线程都在忙，任务队列也满了，新的任务过来的时候会拒绝任务
- 任务拒绝策略：
  -

策略名称	描述
AbortPolicy	默认的拒绝策略， 丢弃任务并会抛出RejectedExecutionException异常
DiscardPolicy	丢弃任务，但是不抛异常
DiscardOldestPolicy	丢弃队列中等待最久的任务，然后把当前任务加入到队列中
CallerRunsPolicy	由主线程负责调用任务的run()方法从而绕过线程池直接运行

  -
    ```
    TE.execute(r); //提交第一个任务 线程池会创建一个线程执行
    TE.execute(r); //提交第二个任务 线程池会创建一个线程执行
    TE.execute(r); //提交第三个任务 线程池会创建一个线程执行
    TE.execute(r); 加入任务等待队列
    TE.execute(r); 加入任务等待队列
    TE.execute(r); 加入任务等待队列，目前为止任务队列已经有三个了，满了，由于Runnable实现类用了sleep所以人工让前三个线程无限忙
    TE.execute(r); 创建临时线程
    TE.execute(r); 创建临时线程，目前为止已经创建了5个线程，线程池已经满了
    TE.execute(r); 又来了新任务，用到了拒绝策略，默认丢弃任务，抛出异常报错
    ```

- **利用线程池处理Callable任务**
-
  ```
  import java.util.concurrent.*;
  public class demo2 {
  public static void main(String[] args) {
  //使用线程池的实现类ThreadPoolExecutor声明7个参数来创建
  ThreadPoolExecutor TE = new ThreadPoolExecutor(
  3,5,
  1000, TimeUnit.MILLISECONDS,
  new ArrayBlockingQueue< Runnable>(3),
  Executors.defaultThreadFactory(),
  new ThreadPoolExecutor.AbortPolicy());
  Future< String> ft1 = TE.submit( new mycallable(100)); 提交任务，返回一个Future对象，即结果
  Future< String> ft2 = TE.submit( new mycallable(200));提交任务，返回一个Future对象，即结果
  Future< String> ft3 = TE.submit( new mycallable(300));提交任务，返回一个Future对象，即结果
  Future< String> ft4 = TE.submit( new mycallable(400));提交任务，返回一个Future对象，即
  ```

```
结果
try {
System.out.println(ft1.get()); 获取结果
System.out.println(ft2.get()); 获取结果
System.out.println(ft3.get()); 获取结果
System.out.println(ft4.get()); 获取结果
} catch (InterruptedException e) {
e.printStackTrace();
} catch (ExecutionException e) {
e.printStackTrace();
}
}
}
import java.util.concurrent.Callable;
public class mycallable implements Callable< String > {
private int n; 任务数
public mycallable(int n) {
this.n = n; 构造方法
}
@Override
public String call() throws Exception {
int sum = 0;
for(int i = 1; i <=n; i++){ 循环累加
sum += i;
}
return Thread.currentThread().getName()+"子线程计算后的结果为："+ sum; 返回结果
}
}
```

6. **并发/并行**
   - 正在运行的程序(软件)就是一个独立的进程
   - 线程是属于进程的，一个进程中可以同时运行多个线程
   - **进程中的多个线程其实是并发和并行执行的**
   - **并发**：进程中的线程是由CPU负责调度的，由于CPU能同时处理线程的数量有限，为了保证全部线程都能往前执行，**CPU(单核)会轮询为系统的每个线程服务，由于CPU切换速度很快，给我们感觉这些线程在同时执行，就是并发**
   - **并行**：在同一时刻上，同时有多个线程在被CPU调度执行。CPU(多核)
   - 实际上是并发并行同时存在，即现代电脑都是多核CPU，并行执行然后并发执行

二、JAVA网络编程

- **网络编程**：可以让设备中的程序与网络上的其他设备中的程序进行数据交互的技术(实现网络通信)
- **基本的通信架构：CS (Client客户端-Server服务端)架构和BS(Browser浏览器-Server服务端)架构**
- CS架构：如微信，IDEA，音乐软件等，需要程序员开发客户端软件，需要用户下载安装客户端软件，服务端需要程序员开发
- BS架构：如百度，淘宝，京东等，需要用户下载安装浏览器，浏览器不需要程序员开发，服务端需要程序员开发
- 无论CS还是BS，都必须依赖网络编程，java.net包提供了网络编程的所有功能

1. **网络编程三要素**
   - **IP地址**：设备在网络中的地址，计算机的唯一标识符，一个IP地址对应一个计算机
   - **端口**：应用程序在设备中的唯一标识
   - **协议**：连接和数据在网络中传输的规则

- **IP地址**：全称"互联网协议地址"，是分配给上网设备的唯一标识，目前被广泛采用的IP地址形式为IPv4，IPv6
  - **IPv4**：是Internet Protocol version 4的缩写， 32位，4字节，使用点分十进制表示：由4个0-255的数字组成，如192.168.1.1。
  - **IPv6**：是Internet Protocol version 6的缩写， 128位，16字节，使用冒分十六进制表示：由8组4位16进制数组成，如2001:0db8:85a3:0000:0000:8a2e:0370:7334。
- **IP域名**：用于在互联网上识别和定位网站的人类可读的名称，如www.baidu.com
- **DNS域名解析器**：将IP域名解析为IP地址，这样才能通过IP地址访问对应的网址
- **公网IP**：是可以连接到互联网的IP地址
- **内网IP**：只能连接局域网的IP地址，是只能组织或机构内部专门使用的IP地址，如192.168.0.0-192.168.255.255，专门用于局域网
- **本机IP**：127.0.0.1 、localhost(这两个是一个意思，这个是域名)，代表本机IP，只会寻找当前程序所在的主机
- **本机IP127.0.0.1指向本机，数据包无法离开我的网卡，无法被外界访问。IPv4的IP地址是在局域网范围内唯一指向本机。公网IP是在互联网范围内唯一指向本机**
- **IP常用命令**：
-

命令	作用
ipconfig	查看本机IP地址，其实就是看IPv4地址， 即局域网开头的192.168开头的IP地址，因为大多数都是局域网
ipconfig /all	查看本机IP地址、DNS、MAC地址等信息
ping 别人的IP地址	测试本机IP地址和别的IP地址的连通性，即检查网络是否联通

- **InetAddress类**：java.net包下的类，用于获取IP地址，如本机IP地址，DNS解析器，IP地址转换，IP地址的比较等
-

方法	作用
getLocalHost()	获取本机IP地址
getByName(String host)	根据IP地址或域名获取InetAddress对象，获取别人的IP地址
getHostName()	获取IP地址对应的主机名
getHostAddress()	获取IP地址
isReachable(int timeout)	判断IP地址是否可达

-
```
public class demo1 {
public static void main(String[] args) {
//认识InetAddress
try{
InetAddress ip1 = InetAddress.getLocalHost(); //获取本机IP
System.out.println(ip1); //LAPTOP-HJJ64TEQ/169.254.172.173,主机名和主机IP地址
System.out.println(ip1.getHostName()); //LAPTOP-HJJ64TEQ
System.out.println(ip1.getHostAddress()); //169.254.172.173 就是自己的IP，即公网IP
InetAddress ip2 = InetAddress.getByName("www.baidu.com"); //获取百度的IP
System.out.println(ip2); //www.baidu.com/39.156.70.239
System.out.println(ip2.getHostName()); //www.baidu.com
System.out.println(ip2.getHostAddress()); //39.156.70.239
System.out.println(ip2.isReachable(1000)); //判断百度是否可以访问，联网了就是true
} catch (UnknownHostException e) {
throw new RuntimeException(e);
} catch (IOException e) {
throw new RuntimeException(e);
}
}
}
```

- **端口**：应用程序在设备中的唯一标识，被规定为一个16位的二进制数。**范围：0-65535，0-1023为系统端口(被预先定义的知名应用占用，如HTTP：占用了80，FTP占用了21等)，1024-49151为用户端口(分配给用户进程或某些应用程序)，49152-65535为动态端口(一般不固定分配某种进程，而是动态分配)**
- **注意，我们自己开发的程序一般使用注册端口，且一个设备中不能出现两个程序的端口号向相同，否则报错**
- **协议**：即通信协议，网络上通信的设备，事先规定的连接规则，以及传输数据的规则，被称为网络通信协议

- **开放式网络互联标准：OSI网络参考模型**：太过于理论
- **TCP/IP网络模型**：TCP/IP网络模型是实际使用的网络模型，即事实上的国际标准，下面是它的层结构

层	各层对应哪些协议	面向操作
应用层	HTTP、FTP、SMTP、POP3、TELNET、DNS	面向应用程序，程序员一般在这里开发
传输层	TCP、UDP	面向连接，选择使用的传输协议
网络层	IP	封装源和目标IP
数据链路层+物理层	ARP、RARP、LLDP、硬件、比特流	物理设备中传输

- **传输层的两个通信协议**
  - **UDP协议**：用户数据报协议，**无连接、不可靠通信**，不事先建立连接，**数据按照包发**，一包数据包含：自己的IP、端口，目的地IP、端口和数据(限制在64KB内)等，发送方不管对方是否在线，数据在中间丢失也不管，接收方收到数据也不返回确认，故不可靠。因此UDP协议适用于**数据量小、快速传输、数据丢失问题不大的情况**，如视频直播，语音通话
  - **TCP协议**：传输控制协议，**面向连接、可靠通信**，其最终目的是要保证在不可靠的信道上实现可靠的数据传输。由三个步骤实现：**三次握手建立连接、传输数据进行确认、四次挥手断开连接**，非常适合做网页、文件下载、支付，因为要求可靠性必须要高
    - **三次握手建立可靠连接**：确保通信双方收发消息都是没问题的(全双工)。**三次握手：客户端向服务端发出连接请求(确保客户端发送消息是成功的)、服务端向客户端返回一个响应(确保服务端发送和接收消息是成功的)、客户端向服务端再次发出确认消息，连接建立(确保客户端接收消息是成功的)**
    - **四次挥手断开连接**：确保通信双方收发消息都已经完成。**四次挥手：客户端向服务端发出断开连接请求，服务端向客户端返回一个响应、稍等，这时服务器将最后的数据处理完后，向客户端返回一个响应：消息处理完毕，确认断开，客户端发出确认断开信息，连接断开(这是客户端等待服务端返回的所有消息处理完，再断开连接)**
- **UDP通信**：**java.net.DatagramSocket类实现UDP通信**

方法	作用
DatagramSocket()	创建一个客户端的Socket对象，并随机分配一个端口
DatagramSocket(int port)	创建一个服务端的Socket对象，并指定端口号
send(DatagramPacket p)	发送数据包
receive(DatagramPacket p)	使用数据包接收数据

方法	作用
DatagramPacket(byte[] buf, int length, InetAddress ip, int port)	创建发出去的数据包对象
DatagramPacket(byte[] buf, int length)	创建用来接收数据的数据包

- import java.io.IOException;
  import java.net.;
  *public class UDPClient { **这是客户端代码***
  public static void main(String[] args) throws IOException {
  System.out.println("UDPClient启动");
  //完成UDP一发一收，客户端开发
  DatagramSocket ds = new DatagramSocket(); //创建发送端对象
  *byte[] bytes = "hello,UDP".getBytes(); **//数据内容***
  DatagramPacket dp = new DatagramPacket(bytes, bytes.length, InetAddress.getLocalHost(),8080); //创建要发的数据报包
  参数一：发送的数据，字节数组 bytes
  参数二：字节数组的长度 bytes.length
  参数三：指定发送的目标地址 InetAddress.getLocalHost(),这是本地IP地址
  参数四：指定发送的目标端口 8080
  *ds.send(dp); **发送数据***
  }
  }
  import java.io.IOException;
  import java.net.DatagramPacket;
  import java.net.DatagramSocket;
  import java.net.SocketException;
  *public class UDPService { **这是服务端代码***
  public static void main(String[] args) throws IOException {
  System.out.println("服务端启动");
  DatagramSocket ds = new DatagramSocket(8080); //创建接收端对象，指定为端口8080，就是客户端数据包那个端口，这样才能匹配上
  *byte[] bytes = new byte[102464]; **创建一个字节数组，用来接收数据，数据大小就是64KB，最大的数据量***
  DatagramPacket dp = new DatagramPacket(bytes, bytes.length); **创建一个数据包对象，用来接收数据**
  ds.receive(dp); **接收数据**
  int length = dp.getLength(); **获取数据长度**
  String data = new String(bytes, 0, length); **把字节数组转换成字符串，并截取有效数据**
  System.out.println(data);
  String ip=dp.getAddress().getHostAddress(); **获取发送端的IP地址**

```
int port=dp.getPort(); 获取发送端的端口号
System.out.println("对方IP为："+ip+" 对方端口为"+port);
}
}先启动服务端，再启动客户端，即可完成数据传输
```

- **UDP通信的多发多收**
- 客户端实现步骤：
  - 创建DatagramSocket对象(发送端对象)
  - 使用while死循环不断地接收用户输入，如果用户输入exit就退出
  - 如果用户输入的不是exit，就创建DatagramPacket对象，封装数据，发送给服务端
  - 释放资源
- 服务端实现步骤：
  - 创建DatagramSocket对象(接收端对象)，并指定端口号
  - 创建DatagramPacket对象，封装数据，接收数据
  - 用receive方法接收数据
  - 使用while死循环，不断运行上一步
-

```
public class UDPClient {
public static void main(String[] args) throws IOException {
System.out.println("UDPClient启动");
//完成UDP一发一收，客户端开发
DatagramSocket ds = new DatagramSocket(); //创建发送端对象
Scanner sc = new Scanner(System.in); 创建键盘输入对象
while(true) { 循环发送数据
System.out.println("请输入要发送的内容：");
String data = sc.nextLine();
if("exit".equals(data)){ 判断输入内容是否为exit
System.out.println("UDPClient退出");
ds.close(); 释放资源
break; 退出循环
}
byte[] bytes = data.getBytes(); //数据内容
/*
参数一：发送的数据，字节数组
参数二：字节数组的长度
参数三：指定发送的目标地址
参数四：指定发送的目标端口
/
DatagramPacket dp = new DatagramPacket(bytes, bytes.length,
InetAddress.getLocalHost(), 8080); //创建要发的数据报包
ds.send(dp);
} 循环结束
```

```
}
} 当然客户端终端可以运行多个，服务端同样可以接收,对于UDP而言可以多次输入，也可以多个终端输入
public class UDPService {
public static void main(String[] args) throws IOException {
System.out.println("服务端启动");
DatagramSocket ds = new DatagramSocket(8080);
byte[] bytes = new byte[102464];
DatagramPacket dp = new DatagramPacket(bytes, bytes.length);
while(true) { 循环接收数据
ds.receive(dp);
int length = dp.getLength();
String data = new String(bytes, 0, length);
System.out.println(data);
String ip = dp.getAddress().getHostAddress();
int port = dp.getPort();
System.out.println("对方IP为：" + ip + " 对方端口为" + port);
}
}
}
```

- **TCP通信的实现**：java.net.Socket类来实现TCP通信
- **TCP通信一发一收**
  - 客户端开发：
  -

方法	作用
Socket(InetAddress ip, int port)	创建一个Socket对象，服务端目标IP和端口
getOutputStream()	获取一个字节输出流，用于发送数据
getInputStream()	获取一个字节输入流，用于接收数据

  - 创建客户端的Socket对象，指定服务端的IP和端口号
  - 使用Socket对象中的方法getOutputStream，获取一个字节输出流
  - 使用字节输出流，写数据到数据通道，可以把字节输出流包装成特殊数据流DataOutputStream，可以方便传出一堆数据类型的数据
  - 释放资源，关闭Socket管道
  -

```
public static void main(String[] args) throws IOException {
Socket socket = new Socket("127.0.0.1", 8080);
OutputStream os = socket.getOutputStream();
DataOutputStream dos = new DataOutputStream(os); //包装成特殊数据流
dos.writeUTF("hello,我是客户端");
```

```
dos.writeInt(100);
socket.close();
System}
```

- 服务端开发：java.net.ServerSocket类来实现
-

方法	作用
ServerSocket(int port)	创建一个ServerSocket对象，指定端口
accept()	监听，接收Socket管道
getInputStream()	获取字节输入流，用于接收数据
getOutputStream()	获取字节输出流，用于发送数据

-

```
public class TCPService {
public static void main(String[] args) throws IOException {
ServerSocket ss = new ServerSocket(8080);
Socket socket = ss.accept();
InputStream is = socket.getInputStream();
DataInputStream dis = new DataInputStream(is); 客户端将数据输出流包装成特殊数据流，则服务端也要用特殊数据流进行接收
String data = dis.readUTF();
int id = dis.readInt(); 客户端先输出整数，再输出字符串，则服务端要先接收整数，再接收字符串
System.out.println("收到客户端数据："+data+" id为："+id);
System.out.println("客户端的IP："+socket.getInetAddress().getHostAddress()+"端口为:"+socket.getPort());
socket.close();
}
}
```

- **TCP通信多发多收**：客户端用死循环，让用户不断输入消息。服务端也使用死循环，接收数据。这个只是可以单个终端连续发送多个数据而已
-

```
public class TCPClient {
public static void main(String[] args) throws IOException { 客户端
Socket socket = new Socket("127.0.0.1", 8080);
OutputStream os = socket.getOutputStream();
DataOutputStream dos = new DataOutputStream(os); //包装成特殊数据流
Scanner sc = new Scanner(System.in);
while(true) { 客户端用死循环，让用户不断输入消息
System.out.println("请输入要发送的内容：");
String data = sc.nextLine();
if("exit".equals(data)){
```

```
System.out.println("TCPClient退出");
socket.close();
break;
}
dos.writeUTF(data);
dos.flush(); 刷新缓冲区
}
}
}
public class TCPService { 服务端
public static void main(String[] args) throws IOException {
ServerSocket ss = new ServerSocket(8080);
Socket socket = ss.accept();
InputStream is = socket.getInputStream();
DataInputStream dis = new DataInputStream(is);
while(true) { 服务端用死循环，接收数据
String data = dis.readUTF();
System.out.println("收到客户端数据："+data);
System.out.println("客户端的IP："+socket.getInetAddress().getHostAddress()+"端口为:"+socket.getPort());
}
}
}
```

- **TCP通信下，同时接收多个客户端通信**
  - 为什么之前的TCP无法同时接收多个客户端通信呢？因为TCP是单线程，即Socket socket = ss.accept();只能获取一个Socket对象，故无法同时处理多个客户端
  - **则需要用到多线程：在服务端：主线程负责接收客户端连接，接下来每来一个客户端，就开启一个独立的子线程来处理这个客户端**
  -

```
import java.io.DataOutputStream;
import java.io.IOException;
import java.io.OutputStream;
import java.net.Socket;
import java.util.Scanner;
public class TCPClient { 客户端，还是原先的代码，不用动
public static void main(String[] args) throws IOException {
Socket socket = new Socket("127.0.0.1", 8080);
OutputStream os = socket.getOutputStream();
DataOutputStream dos = new DataOutputStream(os); //包装成特殊数据流
Scanner sc = new Scanner(System.in);
while(true) {
```

```
System.out.println("请输入要发送的内容：");
String data = sc.nextLine();
if("exit".equals(data)){
System.out.println("TCPClient退出");
socket.close();
break;
}
dos.writeUTF(data);
dos.flush();
}
}
}
import java.io.DataInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.net.ServerSocket;
import java.net.Socket;
import java.security.Provider;
public class TCPService { 服务端，多线程处理
public static void main(String[] args) throws IOException {
ServerSocket ss = new ServerSocket(8080);
while(true) { 主线程负责循环接收客户端连接
Socket socket = ss.accept(); 接收客户端连接
System.out.println("有一个客户端上线了");
System.out.println("客户端的IP为：" + socket.getInetAddress().getHostAddress()); 获取客户端的IP
new ServerReader(socket).start(); 开启一个子线程，处理这个客户端
}
}
}
import java.io.DataInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.net.Socket;
public class ServerReader extends Thread{ 线程类，继承Thread类，重写run方法
private Socket socket; 成员变量，保存Socket对象
public ServerReader(Socket socket) {
this.socket = socket; 构造方法，把外部传入的Socket对象保存
}
@Override
```

```
public void run() { 重写run方法，循环处理客户端数据，因为一个客户端不可能就传一次数据
try {
InputStream is = socket.getInputStream();
DataInputStream dis = new DataInputStream(is);
while (true) { 循环处理数据
String data = dis.readUTF();
System.out.println("收到客户端数据：" + data);
System.out.println("客户端的IP：" + socket.getInetAddress().getHostAddress() + "端口为:" + socket.getPort());
}
} catch (IOException e) {
e.printStackTrace();
System.out.println("一个TCPClient下线了"); 在客户端输出exit时，这边会把错误抛出，并且打印客户端下线了，并且打印客户端的IP和端口，主要是一个提醒作用，更直观
System.out.println("客户端的IP：" + socket.getInetAddress().getHostAddress() + "端口为:" + socket.getPort());
}
}
}
```

- **B/S架构：Browser + Server的原理**
  - 浏览器如何请求服务端：http://服务器IP:服务器端口。如http://127.0.0.1:8080
  - 服务端主线程：接收客户端连接。然后浏览器发一个请求，就与服务端建立一个Socket连接，然后创建一个子线程处理。以此类推
  - **服务器必须给浏览器响应HTTP协议规定的数据格式，否则浏览器不识别返回的数据**
  -

HTTP协议规定返回的数据格式	需要回车换行
协议版本 空格 状态码 空格 状态符	回车换行
HTTP/1.1 200 OK	\r\n
头部字段名：值;	回车换行
Content-Type:text/html; charset=UTF-8	\r\n
.....	回车换行
.....	\r\n
头部字段名：值;	回车换行
Content-Length: 100	\r\n

HTTP协议规定返回的数据格式	需要回车换行
在上述写完头字段后，必须单独换一行， 否则浏览器会认为数据不完整，无法解析	
\r\n	
响应正文(真正给浏览器展示的网页数据)	
< div style='color: red;"font-size: 50px;text-align;center'> 啦啦啦啦这是给浏览器展示的正文，前面一堆是数据的字体颜色， 大小，位置< /div >	

- 只需要修改服务器的线程类代码即可，不用写客户端代码。并且因为涉及浏览器的数据展示，所以有一堆前端知识
-
```
import java.io.DataInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.net.ServerSocket;
import java.net.Socket;
import java.security.Provider;
public class TCPService { 服务端，多线程处理
public static void main(String[] args) throws IOException {
ServerSocket ss = new ServerSocket(8080);
while(true) {
Socket socket = ss.accept();
System.out.println("有一个客户端上线了");
System.out.println("客户端的IP为：" + socket.getInetAddress().getHostAddress());
new ServerReader(socket).start();
}
}
}
import java.io.*;
import java.net.Socket;
public class ServerReader extends Thread{ 线程类，重点修改
private Socket socket;
public ServerReader(Socket socket) {
this.socket = socket;
}
@Override
public void run() {
try {
OutputStream os = socket.getOutputStream(); //字节输出流，把服务端数据推到浏览器
```

> **上展示**
> PrintStream ps = new PrintStream(os); **//因为HTTP格式每次都要换行，所以跟打印流非常契合，因为打印流写数据自带换行**
> **//所以将字节输出流包装为打印流**
> **ps.println("HTTP/1.1 200 OK");**
> **ps.println("Content-Type:text/html; charset=UTF-8");**
> **ps.println(); //最后一行必须要换行**
> **ps.println("< html>");**
> **ps.println("< head>< title>Hello World< /title>< /head>");**
> **ps.println("< body>");**
> **ps.println("< h1 style='color=red;font-size:100px'>Hello World。啦啦啦啦这是给浏览器展示的正文< /h1>");**
> **ps.println("< /body>");**
> **ps.println("< /html>");**
> **ps.close();**
> **socket.close();上面一堆输出的就是前端知识 可以关闭socket和输出流，因为浏览器再打开一个页面就会创建一个新的管道**
> } catch (IOException e) {
> e.printStackTrace();
> System.out.println("一个TCPClient下线了");
> System.out.println("客户端的IP：" + socket.getInetAddress().getHostAddress() + "端口为:" + socket.getPort());
> }
> }
> }**运行服务器代码后，在浏览器输入http://127.0.0.1:8080(因为这是本机传输，并且服务器设置了端口号为8080)，就会看到浏览器展示的你自己想要展示的数据：Hello World。啦啦啦啦这是给浏览器展示的正文**

- 由于B/S架构底层原理，开一个网页就要创建一个新线程，在原先局域网内或许够用，但是浏览器肯定是不够用，而且开了新线程又会很快销毁，所以可以用线程池进行优化
- 即把所有的socket都包装成任务对象runnable，然后交给线程池任务队列，然后用核心线程处理这成百上千的请求
- **只需要修改服务端代码**
-
  import java.io.DataInputStream;
  import java.io.IOException;
  import java.io.InputStream;
  import java.net.ServerSocket;
  import java.net.Socket;
  import java.security.Provider;
  import java.util.concurrent.ArrayBlockingQueue;
  import java.util.concurrent.ExecutorService;

```
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;
public class TCPService { 服务端，线程池处理
public static void main(String[] args) throws IOException {
ServerSocket ss = new ServerSocket(8080);
ExecutorService executorService = new ThreadPoolExecutor(
10,
10,
1000,
TimeUnit.MILLISECONDS,
new ArrayBlockingQueue< Runnable >(10),
new ThreadPoolExecutor.AbortPolicy()); 创建线程池
while(true) {
Socket socket = ss.accept();
System.out.println("有一个客户端上线了");
System.out.println("客户端的IP为：" + socket.getInetAddress().getHostAddress());
executorService.execute(new ServerReader(socket)); 线程池处理，移交runnable任务给线程池处理，线程对象其实本身就是任务，因为隐含有runnable接口
}
}
}
```